Iterative Neural Architecture Search via LLM-Driven Code Generation with Historical Feedback Memory

Name Surname*, Name Surname, Name Surname, Dmitry Ignatov, Radu Timofte
Computer Vision Lab, CAIDAS & IFI, University of Würzburg, Germany

Abstract

Neural Architecture Search (NAS) automates network design but typically demands substantial computational resources. We propose a closed-loop pipeline that leverages a small (4-billion-parameter) large language model (LLM) to iteratively generate, evaluate, and refine convolutional neural network architectures for image classification. Central to our approach is a historical feedback memory *mechanism inspired by Markov chains: a sliding window of the most recent improvement attempts is fed back to the LLM, enabling it to learn from past successes and failures without additional training. Using one-epoch proxy accuracy on CIFAR-10 as a fast architecture ranking signal, our pipeline improves generated architectures from 22.2% to 59.2% top-1 accuracy over 100 successful evaluations (Spearman* $\rho = 0.70$, $p < 10^{-15}$*), representing a* $2.7\times$ *improvement over single-shot generation. These results demonstrate that even small LLMs can serve as effective architecture designers when equipped with iterative feedback and historical memory, offering a lightweight and accessible paradigm for LLM-driven NAS.*

1. Introduction

Neural Architecture Search (NAS) has emerged as a powerful paradigm for automating the design of deep neural networks, achieving competitive or superior performance to hand-crafted architectures across diverse tasks [13, 17, 23]. However, conventional NAS methods—whether based on reinforcement learning [23], evolutionary algorithms [17], or differentiable relaxations [13]—remain computationally expensive, often requiring hundreds or thousands of GPU hours to search a single architecture. This cost has motivated training-free approaches that use proxy metrics to rank architectures without full training [12, 14], yet these methods still operate within predefined, discrete search spaces.

Recent advances in large language models (LLMs) have opened a fundamentally different avenue: using LLMs as architecture generators that produce executable neural network code directly [8, 10, 22]. Unlike traditional NAS, LLM-based approaches operate in the space of *programs* rather than fixed cell-based encodings, enabling more flexible and expressive architecture design. Prior work within the NNGPT framework [10] and related efforts [6, 15, 20] have demonstrated that LLMs can generate functional vision models, but predominantly through single-shot or few-shot generation without iterative self-improvement.

A critical limitation of single-shot LLM generation is that it treats architecture design as a one-pass prediction, discarding the evaluation signal entirely. In contrast, human engineers iterate: they build, test, analyze failures, and refine their designs based on accumulated experience. This observation motivates our central question: *Can a small LLM iteratively improve neural network architectures by learning from a structured history of its own attempts?*

We propose a closed-loop pipeline comprising three components: (1) a **Code Generator** that prompts a 4-billion-parameter LLM to produce PyTorch model implementations, (2) an **Evaluator** that trains each generated model for a single epoch on CIFAR-10 as a fast proxy for architecture quality, and (3) a **Prompt Improver** that analyzes evaluation results alongside a sliding window of recent iteration history—a mechanism we term *historical feedback memory*—to produce targeted improvement suggestions for the next iteration. This design draws on the Markov property: the improvement decision at each step depends on the current best architecture and a bounded window of recent transitions, rather than the full trajectory.

Our main contributions are:

- A closed-loop, iterative NAS pipeline driven by a small (4B-parameter) LLM that progressively discovers better architectures through code generation, evaluation, and prompt refinement.
- A *historical feedback memory* mechanism that maintains a sliding window of past improvement attempts, enabling the LLM to avoid repeating failed strategies and build on successful ones.
- An empirical demonstration that this pipeline improves

*Corresponding author: email@uni-wuerzburg.de

architecture quality from 22.2% to 59.2% one-epoch proxy accuracy on CIFAR-10 over 100 evaluations, with a strong positive trend (Spearman $\rho = 0.70$, $p < 10^{-15}$).

2. Related Work

Neural Architecture Search. NAS methods aim to automate the design of neural network architectures. Early approaches employed reinforcement learning [23] and evolutionary algorithms [17] to explore architecture search spaces, but required thousands of GPU hours per search. Parameter-sharing methods such as ENAS [16] and differentiable approaches like DARTS [13] substantially reduced search cost by amortizing evaluation across architectures. More recently, *zero-shot* or *training-free* NAS methods [12, 14] bypass training entirely by scoring architectures at initialization using proxy metrics such as activation overlap [14] or gradient-based indicators. While effective within their respective search spaces, these methods remain constrained to predefined, discrete architecture parameterizations (e.g., cell-based structures) and cannot generate truly novel architectural patterns.

LLMs for Code Generation and AutoML. Large language models have demonstrated strong capabilities in code generation [2], and their application to machine learning automation is an active research area. The NNGPT framework [10] pioneered the use of LLMs as neural network generators, showing that language models can produce functional vision models from textual prompts. Subsequent work explored fractal-inspired architectures [15], few-shot prompting strategies [22], novel architecture creativity [8], and non-standard channel priors [20]. LLMs have also been applied to hyperparameter optimization [9], data transformation design [19], and collaborative vision-language pipelines [4, 7, 18]. These efforts are supported by curated datasets of neural network architectures [5, 21] and deployment pipelines [3]. However, most existing approaches treat LLM-based architecture generation as a single-step process, without leveraging iterative refinement from evaluation feedback. Our work extends this line by introducing a closed-loop pipeline with historical memory that enables progressive architecture improvement.

3. Methodology

Inspired by recent advancements in the application of LLMs across various domains [4, 7, 9, 18] and prior architectural synthesis experiments within the NNGPT framework [6, 8, 9, 15, 19, 20, 22], and leveraging the existing LEMUR dataset of a broad range of high-capacity and edge-optimized models [3, 5, 21], we developed an iterative NAS pipeline based on the NNGPT framework [10]. The pipeline consists of three core modules operating in a closed loop, as illustrated in Figure 1.

3.1. Pipeline Overview

At each iteration t, the pipeline executes the following cycle: (1) the Code Generator produces a candidate architecture $\mathcal{A}_t$ as executable PyTorch code; (2) the generated code is validated and, if structurally correct, trained and evaluated by the Evaluator to obtain a proxy accuracy a_t; (3) the Prompt Improver analyzes the result in the context of the current best architecture and recent iteration history, producing improvement suggestions s_t for the next iteration. The best-performing architecture $\mathcal{A}^* = \arg\max_{i \le t} a_i$ is maintained as the reference implementation throughout the search.

3.2. Code Generator

The Code Generator uses a pre-trained LLM (Qwen3-4B-Instruct, a 4-billion-parameter instruction-tuned model) to produce a complete PyTorch model class implementing `nn.Module`. The LLM receives a fixed prompt template comprising: (i) a role description positioning the model as a "visionary deep learning architect," (ii) the task specification (CIFAR-10 image classification without pre-trained weights), (iii) the current best implementation $\mathcal{A}^*$ as reference code, and (iv) the improvement suggestions s_{t-1} from the previous iteration. The generated code must define a `Net(nn.Module)` class with standard `__init__` and `forward` methods. Generation uses temperature $\tau = 0.7$ and nucleus sampling ($p = 0.9$) to balance architectural diversity with code coherence.

3.3. Evaluator

Generated code undergoes a two-stage evaluation process.

Quick validation. The model is instantiated and a forward pass is performed with a dummy input tensor of shape $2 \times 3 \times 32 \times 32$. The output shape is verified to match $B \times 10$ (for batch size B and 10 CIFAR-10 classes). Models failing this check are discarded with an error message forwarded to the Prompt Improver.

Proxy training. Models passing validation are trained for **one epoch** on CIFAR-10 using SGD (momentum 0.9, weight decay 5×10^{-4}), an initial learning rate of 0.01 with cosine annealing, and a batch size of 128. Standard data augmentation is applied: random crop with 4-pixel padding and random horizontal flip. Top-1 test accuracy on the 10,000-image test set serves as the proxy metric. Training is executed in an isolated subprocess with a 30-minute timeout. One-epoch accuracy provides a fast, informative signal for ranking architectures [14], enabling rapid iteration without the cost of full convergence training.

Algorithm 1: Iterative LLM-Driven NAS Pipeline

Input: LLM $\mathcal{L}$, max iterations T, history window K

$\mathcal{A}^* \leftarrow \emptyset$; $a^* \leftarrow 0$; $\mathcal{H} \leftarrow \emptyset$; $s_0 \leftarrow \emptyset$;
for $t \leftarrow 1$ **to** T **do**
  $\mathcal{A}_t \leftarrow \text{Generate}(\mathcal{L}, \mathcal{A}^*, s_{t-1})$;
  if *Validate*$(\mathcal{A}_t)$ *succeeds* **then**
    $a_t \leftarrow \text{TrainAndEvaluate}(\mathcal{A}_t)$;
  else
    $a_t \leftarrow$ error message;
  if $a_t > a^*$ **then**
    $\mathcal{A}^* \leftarrow \mathcal{A}_t$; $a^* \leftarrow a_t$;
  Append (s_{t-1}, a_t) to $\mathcal{H}$; keep last K entries;
  $s_t \leftarrow \text{Improve}(\mathcal{L}, \mathcal{A}^*, \mathcal{A}_t, a_t, \mathcal{H})$;
return $\mathcal{A}^*$, a^*

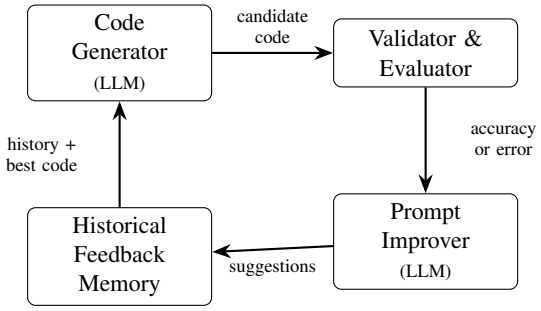

Figure 1. Overview of the iterative NAS pipeline. The Code Generator produces candidate architectures as executable PyTorch code. The Evaluator validates and trains them using one-epoch proxy evaluation. The Prompt Improver analyzes results with historical feedback memory to generate targeted improvement suggestions for the next iteration.

3.4. Prompt Improver with Historical Feedback Memory

The Prompt Improver is the central component enabling iterative learning. After each evaluation, it receives three inputs: (1) the best code $\mathcal{A}^*$ and its accuracy a^*, (2) the current iteration's code $\mathcal{A}_t$ and its evaluation outcome (accuracy value or error message), and (3) a *historical feedback memory* $\mathcal{H}_t$ containing the last K=5 improvement attempts.

Each history entry $h_i \in \mathcal{H}_t$ is a triple recording: the identified problem, the suggested improvement, and the resulting outcome (accuracy achieved or error encountered). This sliding-window design draws on the Markov property: the improvement decision at step t depends on the current state (best architecture plus recent history window) rather than the complete trajectory. The bounded window prevents context overflow while providing sufficient signal for the LLM to identify recurring failure patterns.

The Prompt Improver produces a structured response containing: (a) a **reason** diagnosing the current result, (b) an **inspiration** drawing on cross-disciplinary insights, and (c) concrete **improvement suggestions** for the next iteration. These suggestions, together with the best code, form the input to the Code Generator in the subsequent iteration, closing the feedback loop.

The overall procedure is summarized in Algorithm 1.

4. Experiments

4.1. Experimental Setup

Dataset. We use CIFAR-10 [11], comprising 60,000 32×32 color images across 10 classes (50,000 training, 10,000 test). Standard augmentation (random crop with 4-pixel padding, random horizontal flip, normalization) is applied during training, inspired by [1].

Language model. Architecture generation and prompt improvement both use Qwen3-4B-Instruct, a 4-billion-parameter instruction-tuned language model. Generation parameters: temperature 0.7, nucleus sampling $p = 0.9$, maximum 2,048 new tokens. A fixed seed with incremental call counter ensures reproducible but diverse generation across iterations.

Training protocol. Each candidate architecture is trained for one epoch with SGD (momentum 0.9, weight decay 5×10^{-4}), learning rate 0.01 with cosine annealing, and batch size 128. A fixed random seed (43) and deterministic CUDA operations ensure reproducibility. Training runs in isolated subprocesses with a 30-minute timeout. Fine-tuning of LLMs and training of computer vision models are performed using the AI Linux docker image `abrainone/ai-linux`[1] on NVIDIA GeForce RTX 3090/4090 24G GPUs of the CVL Kubernetes cluster at the University of Würzburg.

Baseline. We compare against **single-shot generation**: the accuracy of the first successfully generated and evaluated architecture (iteration 1), representing what the LLM produces from its pre-trained knowledge alone without any feedback signal.

4.2. Evaluation Metrics

We report top-1 test accuracy after one-epoch training as the primary architecture quality metric. To characterize search dynamics, we compute: Spearman rank correlation ρ between iteration index and accuracy (trend strength), linear regression slope (improvement rate per 10 iterations), coefficient of determination R^2, and descriptive statistics (mean $\pm$ SD) for early and late evaluation blocks.

[1] AI Linux: https://hub.docker.com/r/abrainone/ai-linux

Table 1. Summary statistics for the iterative NAS pipeline on CIFAR-10 (one-epoch proxy accuracy). The pipeline achieves a $2.7\times$ improvement over single-shot generation.

Metric	**Value**
Successful evaluations	100
Mean accuracy (all)	0.509 ± 0.109
Median accuracy	0.555
Best accuracy (iteration)	0.592 (iter. 65)
Single-shot accuracy (iter. 1)	0.222
First 5 evals (mean $\pm$ SD)	0.258 ± 0.034
Last 5 evals (mean $\pm$ SD)	0.539 ± 0.047
Improvement (last $-$ first 5)	$+0.281$
Spearman ρ	0.705 ($p < 10^{-15}$)
Linear slope (per 10 iter.)	$+0.023$
R^2	0.365

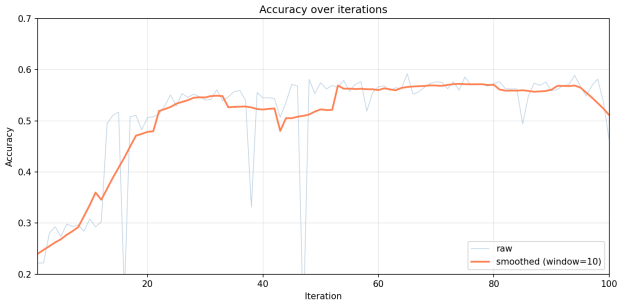

Figure 2. One-epoch proxy accuracy on CIFAR-10 across 100 successful architecture evaluations. The pipeline exhibits three phases: rapid improvement ($\sim$22% to $\sim$50%), refinement ($\sim$50% to $\sim$59%), and plateau ($\sim$55–59%). Spearman $\rho = 0.70$ ($p < 10^{-15}$).

4.3. Main Results

Table 1 summarizes the key results over 100 successful architecture evaluations. The pipeline improved one-epoch proxy accuracy from an initial 22.2% (single-shot baseline) to a peak of 59.2% at iteration 65, with the last 5 evaluations averaging $53.9\% \pm 4.7\%$. The overall mean accuracy across all 100 evaluations is $50.9\% \pm 10.9\%$, with a median of 55.5%.

The Spearman correlation between iteration index and accuracy is $\rho = 0.705$ ($p < 10^{-15}$), indicating a strong, statistically significant positive trend. The linear regression slope of $+2.3$ percentage points per 10 iterations (with $R^2 = 0.365$) confirms steady improvement throughout the search. Compared to single-shot generation (22.2%), the iterative pipeline achieves a $2.7\times$ improvement in best accuracy and a $2.4\times$ improvement in late-phase mean accuracy.

Figure 2 shows the accuracy trajectory across iterations. The curve exhibits three characteristic phases: (1) a *rapid improvement* phase (iterations 1–15) where accuracy jumps from $\sim$22% to $\sim$50% as the LLM quickly discovers effective basic patterns such as convolutional feature hierarchies; (2) a *refinement* phase (iterations 15–65) with continued but slower gains as the model explores deeper architectural modifications; and (3) a *plateau* phase (iterations 65–100) where accuracy stabilizes in the 55–59% range, suggesting diminishing returns from further prompt-driven refinement under the one-epoch training constraint.

4.4. Failure Analysis

Of the approximately 1,000 total generation attempts across the experimental run, roughly 60% failed before reaching the training stage. We categorize failures into three types:

- **Code validation errors** (47.2%): The generated model fails structural checks—most commonly `AttributeError` (referencing undefined layers or modules) and tensor shape mismatches between layers.
- **Code extraction errors** (12.7%): The LLM output cannot be parsed as valid Python, typically due to syntax errors, unclosed string literals, or malformed code blocks.
- **Other runtime errors** (<2%): Errors during training execution, such as device mismatches or batch normalization failures.

The dominant failure mode—`AttributeError`—occurs when the LLM hallucinates layer names or forgets to define modules referenced in the forward pass. Despite the approximately 40% overall success rate, the pipeline's resilience (discarding invalid attempts and continuing) ensures that successfully evaluated architectures exhibit consistent improvement across iterations.

5. Discussion

Effectiveness of iterative refinement. The strong positive correlation ($\rho = 0.70$) across 100 evaluations confirms that historical feedback memory enables systematic architectural improvement. Access to the last K=5 improvement attempts provides the LLM with sufficient context to identify recurring failure patterns and avoid previously unsuccessful strategies. Notably, the three-phase convergence behavior (rapid improvement, refinement, plateau) mirrors patterns in traditional optimization, suggesting that the LLM-driven search exhibits meaningful optimization dynamics rather than random exploration.

One-epoch proxy as architecture ranking signal. Using one-epoch accuracy as a ranking signal offers a pragmatic trade-off between evaluation cost and informativeness. While the absolute accuracy values (22–59%) are well below state-of-the-art CIFAR-10 results with full training (exceeding 95%), prior work on training-free NAS [12, 14] has established that cheap proxy metrics can effectively

rank architectures. Our one-epoch proxy serves a similar role: it distinguishes between architectures of varying quality at minimal cost, enabling the feedback loop to guide the LLM toward better designs. Retraining the best-discovered architectures with full training schedules could verify whether the one-epoch ranking transfers to converged performance.

Limitations. Our study has several limitations. First, we report a single experimental run with one LLM (Qwen3-4B-Instruct); multi-run experiments and cross-model comparisons would strengthen generalizability. Second, the one-epoch proxy may not perfectly correlate with full-training performance; retraining top architectures to convergence is needed to assess their true potential. Third, the $\sim 40\%$ code generation success rate indicates that current LLMs still struggle with structurally valid neural network code, suggesting opportunities for improved prompting strategies or constrained decoding. Finally, our evaluation is limited to CIFAR-10; experiments on larger-scale benchmarks and diverse tasks would better demonstrate versatility.

Future work. Promising directions include: (1) retraining the top-K discovered architectures with full training schedules to verify their converged performance; (2) comparing across LLMs of varying sizes and families; (3) integrating zero-shot proxy metrics [12, 14] as complementary evaluation signals to further accelerate the search; and (4) extending to larger datasets (e.g., ImageNet) and tasks beyond image classification.

6. Conclusion

We presented an iterative NAS pipeline that leverages a small (4B-parameter) LLM to progressively generate and improve neural network architectures through a closed loop of code generation, evaluation, and prompt refinement with historical feedback memory. Over 100 successful evaluations on CIFAR-10, the pipeline improved one-epoch proxy accuracy from 22.2% to 59.2% (Spearman $\rho = 0.70$, $p < 10^{-15}$), demonstrating that iterative feedback with bounded historical memory significantly outperforms single-shot generation. Our results establish that small LLMs can serve as effective architecture search agents when equipped with structured iterative feedback, offering a lightweight and accessible approach to neural architecture search.

References

[1] Nada Aboudeshish, Dmitry Ignatov, and Radu Timofte. Augmentgest: Can random data cropping augmentation boost gesture recognition performance? *arXiv preprint*, arXiv:2506.07216, 2025. 3

[2] Mark Chen et al. Evaluating large language models trained on code. *arXiv preprint*, arXiv:2107.03374, 2021. 2

[3] Saif U Din, Muhammad Ahsan Hussain, Mohsin Ikram, Dmitry Ignatov, and Radu Timofte. Ai on the edge: An automated pipeline for pytorch-to-android deployment and benchmarking. *Preprints*, 2025. 2

[4] Mohamed Gado, Towhid Taliee, Muhammad Danish Memon, Dmitry Ignatov, and Radu Timofte. Vist-gpt: Ushering in the era of visual storytelling with llms? *arXiv preprint*, arXiv:2504.19267, 2025. 2

[5] Arash Torabi Goodarzi, Roman Kochnev, Waleed Khalid, Furui Qin, Tolgay Atinc Uzun, Yashkumar Sanjaybhai Dhameliya, Yash Kanubhai Kathiriya, Zofia Antonina Bentyn, Dmitry Ignatov, and Radu Timofte. LEMUR Neural Network Dataset: Towards Seamless AutoML. *arXiv preprint*, arXiv:2504.10552, 2025. 2

[6] Krunal Jesani, Dmitry Ignatov, and Radu Timofte. Llm as a neural architect: Controlled generation of image captioning models under strict api contracts. *arXiv preprint*, arXiv:2512.14706, 2025. 1, 2

[7] Waleed Khalid, Dmitry Ignatov, and Radu Timofte. A retrieval-augmented generation approach to extracting algorithmic logic from neural networks. *arXiv preprint*, arXiv:2512.04329, 2025. 2

[8] Waleed Khalid, Dmitry Ignatov, and Radu Timofte. From memorization to creativity: Llm as a designer of novel neural-architectures. *arXiv preprint*, arXiv:2601.02997, 2026. 1, 2

[9] Roman Kochnev, Arash Torabi Goodarzi, Zofia Antonina Bentyn, Dmitry Ignatov, and Radu Timofte. Optuna vs Code Llama: Are LLMs a New Paradigm for Hyperparameter Tuning? In *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*, pages 5664–5674, 2025. 2

[10] Roman Kochnev, Waleed Khalid, Tolgay Atinc Uzun, Xi Zhang, Yashkumar Sanjaybhai Dhameliya, Furui Qin, Chandini Vysyaraju, Raghuvir Duvvuri, Avi Goyal, Dmitry Ignatov, and Radu Timofte. Nngpt: Rethinking automl with large language models. *arXiv preprint*, arXiv:2511.2033, 2025. 1, 2

[11] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009. 3

[12] Guihong Li et al. Zero-shot neural architecture search: Challenges, solutions, and opportunities. *IEEE TPAMI*, 46(6): 4071–4088, 2024. 1, 2, 4, 5

[13] Hanxiao Liu, Karen Simonyan, and Yiming Yang. DARTS: Differentiable architecture search. In *ICLR*, 2019. 1, 2

[14] Joseph Mellor, Jack Turner, Amos Shersby, and Elliot J. Shersby. Neural architecture search without training. In *ICML*, pages 7588–7598, 2021. 1, 2, 4, 5

[15] Yash Mittal, Dmitry Ignatov, and Radu Timofte. Preparation of fractal-inspired computational architectures for advanced large language model analysis. *arXiv preprint*, arXiv:2511.07329, 2025. 1, 2

[16] Hieu Pham, Melody Guan, Barret Zoph, Quoc V. Le, and Jeff Dean. Efficient neural architecture search via parameter sharing. In *ICML*, pages 4095–4104, 2018. 2

[17] Esteban Real, Alok Aggarwal, Yanping Huang, and Quoc V. Le. Regularized evolution for image classifier architecture search. In *AAAI*, pages 4780–4789, 2019. 1, 2

[18] Bhavya Rupani, Dmitry Ignatov, and Radu Timofte. Exploring the collaboration between vision models and llms for enhanced image classification. *Preprints*, 2025. 2

[19] Usha Shrestha, Dmitry Ignatov, and Radu Timofte. From brute force to semantic insight: Performance-guided data transformation design with llms. *arXiv preprint*, arXiv:2601.03808, 2026. 2

[20] Tolgay Atinc Uzun, Dmitry Ignatov, and Radu Timofte. Closed-loop llm discovery of non-standard channel priors in vision models. *arXiv preprint*, arXiv:2601.08517, 2026. 1, 2

[21] Tolgay Atincand Uzun, Waleed Khalid, Saif U Din, Sai Revanth Mulukuledu, Akashdeep Singh, Chandini Vysyaraju, Raghuvir Duvvuri, Avi Goyal, Yashkumar Rajeshbhai Lukhi, Ahsan Hussain, Krunal Jesani, Usha Shrestha, Yash Mittal, Roman Kochnev, Pritam Kadam, Mohsin Ikram, Harsh Rameshbhai Moradiya, Alice Arslanian, Dmitry Ignatov, and Radu Timofte. Lemur 2: Unlocking neural network diversity for ai. *arXiv preprint*, 2026. 2

[22] Chandini Vysyaraju, Raghuvir Duvvuri, Avi Goyal, Dmitry Ignatov, and Radu Timofte. Enhancing llm-based neural network generation: Few-shot prompting and efficient validation for automated architecture design. *arXiv preprint*, arXiv:2512.24120, 2025. 1, 2

[23] Barret Zoph and Quoc V. Le. Neural architecture search with reinforcement learning. In *ICLR*, 2017. 1, 2